

Computer Hardware

A Brief History of Concurrency: From a Button to a Scheduler

Ksawery Moźdzynski

Agenda For Today

1. Interrupts

The device tells the processor that something happened

2. Interrupts in the code

`attachInterrupt, IRAM_ATTR, volatile`

3. Interrupts inside the processor

Registers, the stack, `mepc`: what the CPU saves

4. Timer interrupts

A stopwatch with an alarm, a regular tick

5. The scheduler

One processor, several tasks

6. FreeRTOS

A real scheduler on the ESP32-C3

Today you will need

Wokwi



wokwi.com

your project Circuit from Lecture 5

Simulators

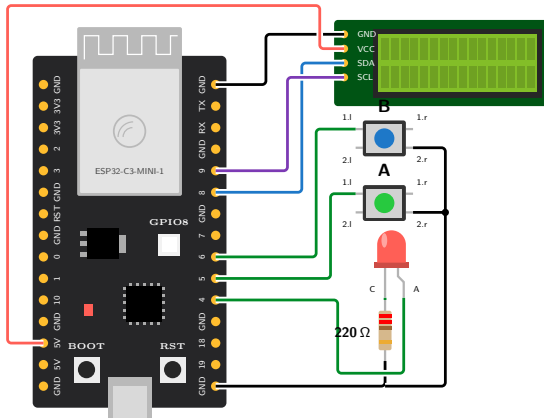


ksawerym.github.io/ComputerHardwareSimulators

No Circuit project from Lecture 5? Download it from GitHub:

github.com/KsaweryM/Computer-Hardware-Course/tree/main/Lecture_6_A_Brief_History_of_Concurrency/Circuit

From last time: our circuit



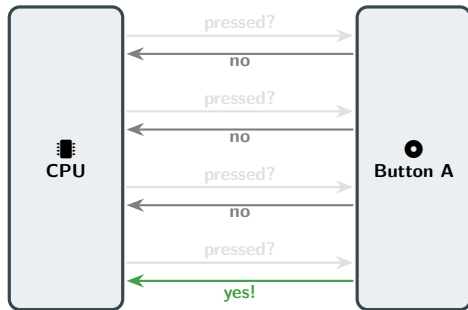
Part	Pin
LED	GPIO 4
Button A	GPIO 5
Button B	GPIO 6
LCD (SDA, SCL)	GPIO 8, 9

Buttons

Pressed = 0, not pressed = 1.

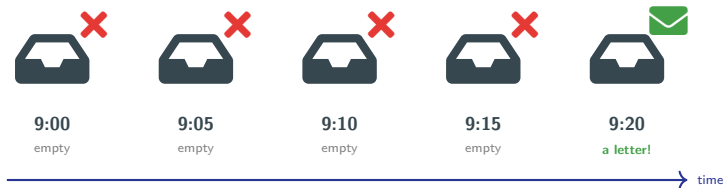
From last time: what is polling?

Polling: the CPU reads the device again and again until something happens.



Polling is simple: one loop that keeps **asking** the button.

Polling is like checking your mailbox



You walk to the mailbox again and again. Almost every time it is **empty**.

The CPU does the same with the button: it walks to it and looks, again and again.

How often should the CPU check the button?



From time to time

Do some work, then check once, then work again.



All the time

Do nothing else: check, check, check.

We tried both last time. Let us look at each one.

Check from time to time: the code

```
● ● ●  
int count = 0;  
  
void loop() {  
    delay(2000);           // "work": 2 s  
    if (digitalRead(BUTTON_A) == LOW) {  
        count++;          // check once  
    }  
    lcd.setCursor(0, 1);  
    lcd.print(count);  
}
```

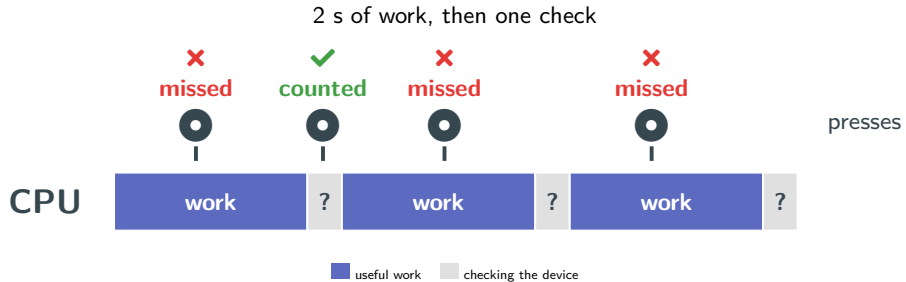
Work

delay(2000) stands for 2 s of other work.

One check

Then the program reads the button **once**, and goes back to work.

Check from time to time: what happens?



A press counts only if the button is down **exactly** when the CPU checks it (green).
Short presses during the work are **lost** (red).

Check from time to time: we miss presses



Presses are lost

Last time

We pressed button A 5 times, shortly. The LCD showed fewer than 5, often 0 or 1.

Why

A short press can come and go **between two checks**. The CPU never sees it.

Checking rarely leaves time for work, but the program **misses events**.

Check all the time: the code

```
● ● ●  
int last = HIGH;    // the last state  
long checks = 0, presses = 0;  
  
void loop() {  
    int now = digitalRead(BUTTON_A);  
    checks++;  
    if (last == HIGH && now == LOW) {  
        presses++;    // a new press  
    }  
    last = now;  
}
```

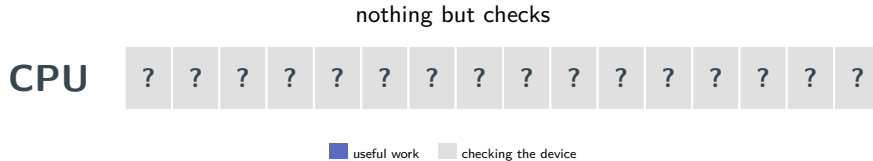
No work

loop() does nothing but read the button.

A new press

It counts a press when the button was HIGH last time and is LOW now.

Check all the time: what happens?



Every press is noticed, but almost every check finds **nothing**.

Check all the time: we waste the CPU



The CPU is wasted

Last time

In 10 s the CPU checked the button hundreds of thousands of times. More than **99.99%** of the checks found nothing.

Why

All the CPU time goes to asking. There is **no time left** for real work.

Checking all the time misses nothing, but it **wastes the CPU**.

Neither way is good



From time to time

time for work, but **presses are lost**



All the time

no press is lost, but **the CPU is wasted**

So what now?

How can the CPU do its work **and** never miss a press?

The problem: with polling the CPU has to **ask**. It cannot know when the button is pressed.

The solution: interrupts

Polling: the CPU asks



Interrupt: the button tells the CPU

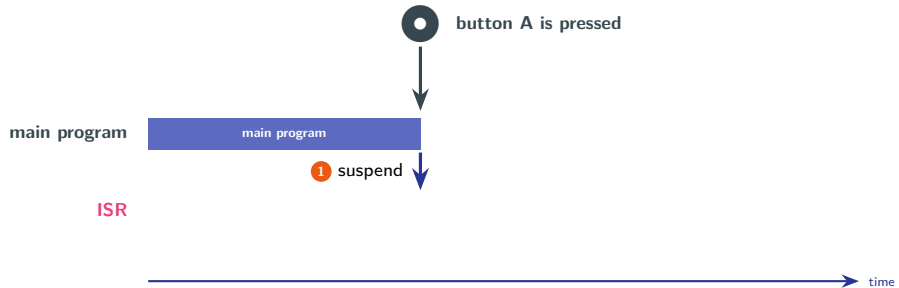


The solution is an **interrupt**: the device itself tells the processor that something happened. Until then, the CPU does its own work.

No missed presses, and no time wasted on asking. Let us see how it works.

An interrupt: 1. the signal

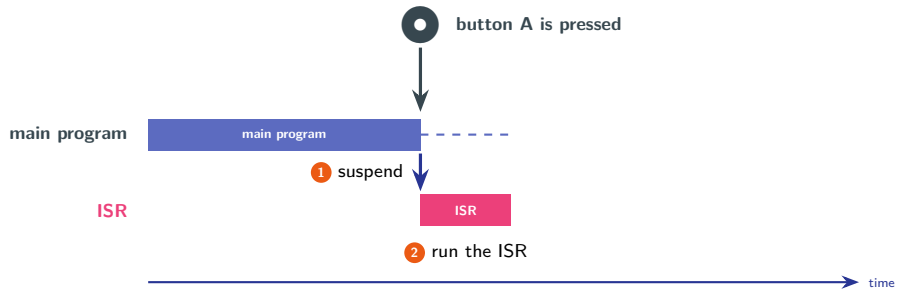
An **interrupt** is a signal from a device to the processor: *something happened, deal with it now.*



1 Button A is pressed. The processor **suspends** the main program at once.

An interrupt: 2. the ISR runs

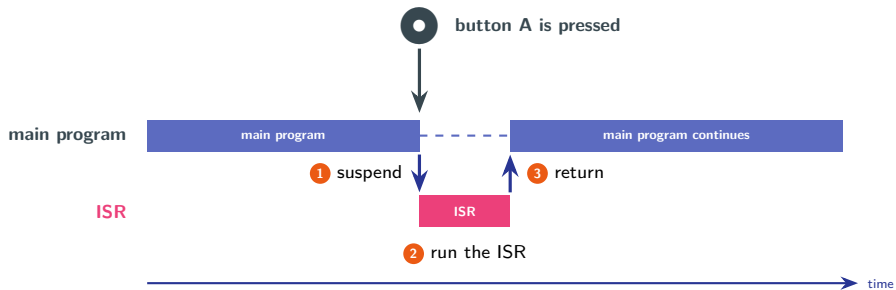
ISR = Interrupt Service Routine: a short function that the processor runs when the interrupt arrives.



2 The processor runs the ISR, for example `presses++`. Meanwhile the main program waits.

An interrupt: 3. return

When the ISR ends, the processor goes back to the main program, **exactly where it stopped**.



3 The main program continues. It **does not know** that it was interrupted.

Try it yourself: an interrupt

Simulators



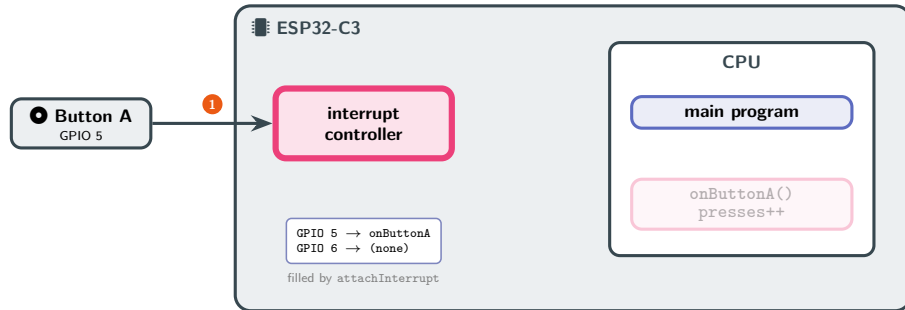
ksawerym.github.io/ComputerHardwareSimulators
topic **An interrupt**

1. Click **Start**. The main program counts: watch Work on the LCD.
2. Press button **A**. Watch the three steps: **suspend, run the ISR, return**. What happens to Work meanwhile?
3. Turn off *slow the ISR down*. How much of the CPU time does the ISR take now?

In the simulator the ISR is slowed down, so that you can see it. Really it takes a few μs .

 ksawerym.github.io/ComputerHardwareSimulators/#cc-intr

Handling an interrupt: 1. the pin changes



- 1 Button A pulls GPIO 5 from 1 to 0 (a FALLING edge). A part of the chip, the **interrupt controller**, notices it. The CPU does not have to check anything.

The interrupt controller has a table

Interrupt	From	Function to run
GPIO 5	button A	onButtonA
GPIO 6	button B	(none)
...		

One row for each interrupt

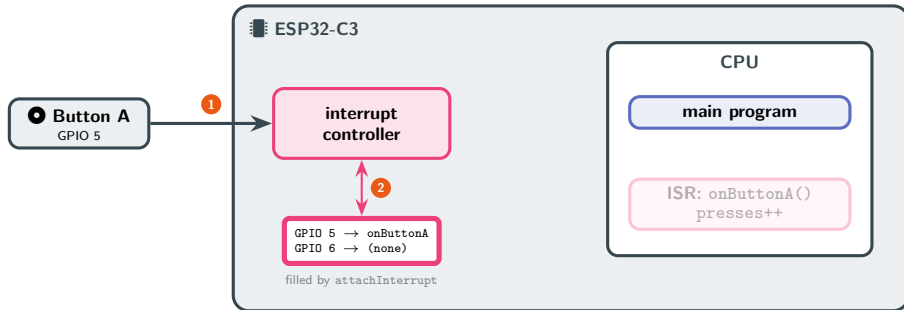
For every interrupt, the table says which function the CPU must run when it arrives.

Who fills it?

Our program, once at the start, with `attachInterrupt`.

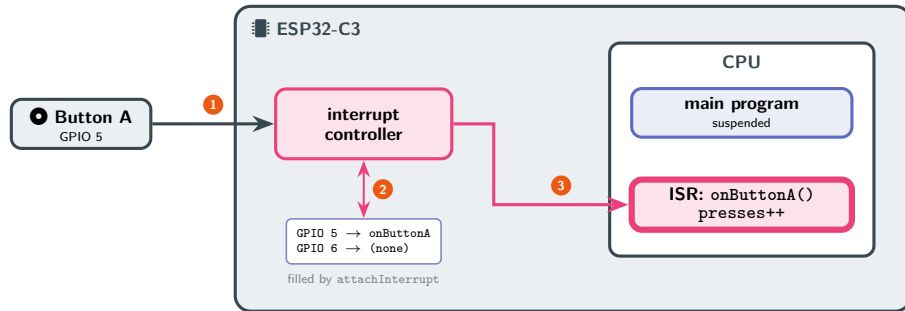
Such a function is called an **ISR**, an *Interrupt Service Routine*: the function that **handles** an interrupt. Nobody calls it in the program: the processor runs it by itself when the interrupt arrives.

Handling an interrupt: 2. which ISR?



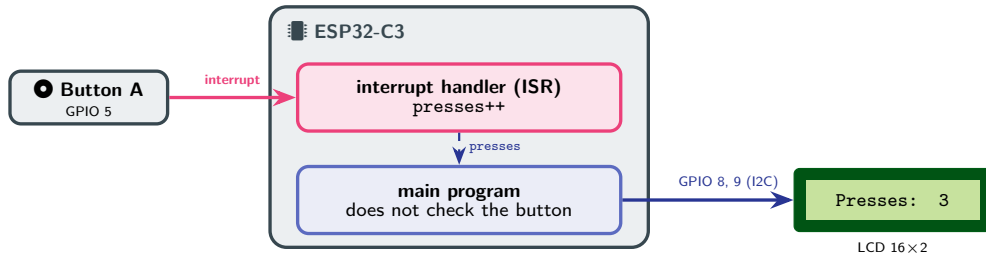
- 2 The interrupt controller looks up GPIO 5 in its table. The ISR for it is `onButtonA`.

Handling an interrupt: 3. the CPU runs the ISR



- 3 The CPU suspends the main program and runs the ISR `onButtonA: presses++`. When the ISR ends, the main program continues.

The button with an interrupt: the main program



The main program **never checks** the button. It only shows how many presses the ISR counted.

Interrupts in the code



```
const int BUTTON_A = 5;
volatile int presses = 0;

void IRAM_ATTR onButtonA() {    // the ISR
    presses++;
}

void setup() {
    pinMode(BUTTON_A, INPUT_PULLUP);
    attachInterrupt(digitalPinToInterrupt(BUTTON_A),
                    onButtonA, FALLING);
}
```

The whole program

The ISR onButtonA counts the presses. setup() connects it to button A.

Four new things

attachInterrupt, digitalPinToInterrupt, IRAM_ATTR and volatile. Let us look at them **one by one**.

Interrupts in the code: attachInterrupt



```
const int BUTTON_A = 5;
volatile int presses = 0;

void IRAM_ATTR onButtonA() {    // the ISR
    presses++;
}

void setup() {
    pinMode(BUTTON_A, INPUT_PULLUP);
    attachInterrupt(digitalPinToInterrupt(BUTTON_A),
                   onButtonA, FALLING);
}
```

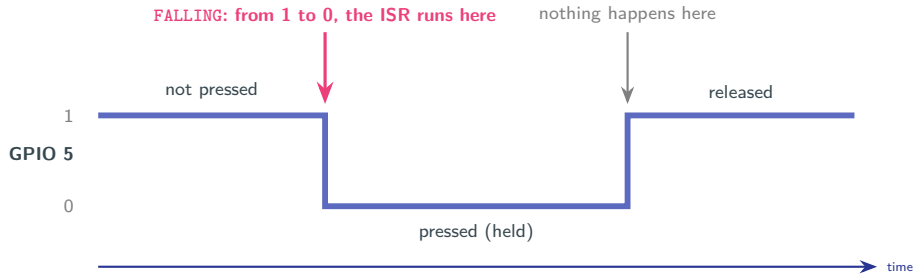
Fill the table

It writes one row into the table of the interrupt controller:

which interrupt, **which** ISR (onButtonA) and **when** (FALLING: the press).

After this line the CPU never calls onButtonA itself: the interrupt does.

Interrupts in the code: FALLING



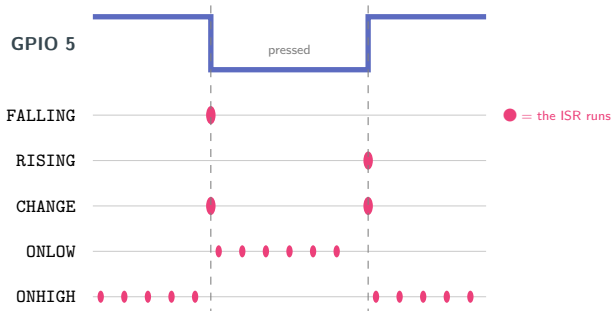
Why 1 and 0?

INPUT_PULLUP: the pin reads **1** when the button is free.
A press connects it to GND: **0**.

An edge, not a level

The ISR runs **once**, at the moment the level falls.
Holding the button longer does not count again.

Interrupts in the code: the other modes



Edges

FALLING: $1 \rightarrow 0$, the press.

RISING: $0 \rightarrow 1$, the release.

CHANGE: both, so one click gives **two** interrupts.

Levels

ONLOW, ONHIGH: the ISR runs **again and again** while the pin stays 0 (or 1).

For a button we want **one** interrupt per press: FALLING.

Interrupts in the code: digitalPinToInterrupt

```
● ● ●  
const int BUTTON_A = 5;  
volatile int presses = 0;  
  
void IRAM_ATTR onButtonA() {    // the ISR  
    presses++;  
}  
  
void setup() {  
    pinMode(BUTTON_A, INPUT_PULLUP);  
    attachInterrupt(digitalPinToInterrupt(BUTTON_A),  
                    onButtonA, FALLING);  
}
```

A pin number → an interrupt number

attachInterrupt wants the number of the **interrupt**, not of the pin. On the ESP32 both are the same; on an Arduino Uno, pin 2 is interrupt 0. With this function the code works everywhere.

Interrupts in the code: IRAM_ATTR



```
const int BUTTON_A = 5;
volatile int presses = 0;

void IRAM_ATTR onButtonA() {    // the ISR
    presses++;
}

void setup() {
    pinMode(BUTTON_A, INPUT_PULLUP);
    attachInterrupt(digitalPinToInterrupt(BUTTON_A),
                    onButtonA, FALLING);
}
```

Keep the ISR in fast memory

The program normally lives in the **flash**: big, slow, and sometimes busy. An ISR must start **at once**, so IRAM_ATTR puts it into **IRAM**, fast memory inside the chip.

Every ISR on the ESP32 gets IRAM_ATTR.

Interrupts in the code: volatile

```
● ● ●  
const int BUTTON_A = 5;  
volatile int presses = 0; ←  
  
void IRAM_ATTR onButtonA() {    // the ISR  
    presses++;  
}  
  
void setup() {  
    pinMode(BUTTON_A, INPUT_PULLUP);  
    attachInterrupt(digitalPinToInterrupt(BUTTON_A),  
                    onButtonA, FALLING);  
}
```

Read it from memory every time

loop() never changes presses, so the compiler may read it **once** into a register and never look again. But the ISR changes it behind the program's back. volatile forbids this shortcut.

Every variable that an ISR and the main program share must be volatile.

Task 1: count the presses



```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);

const int BUTTON_A = 5, BUTTON_B = 6;
volatile int presses = 0;
// TODO: the ISRs of A and B

void setup() {
  Wire.begin(8, 9);
  lcd.init(); lcd.backlight();
  // TODO: the pins and the interrupts
}

void loop() {
  // TODO: show "Presses: N"
}
```

Task

Open your project **Circuit** from Lecture 5 or the template from [GitHub](#). Write the code for points 1 to 3 in place of the TODO lines.

1. Write the ISR of button A: it adds 1 to presses. In setup(), attach it to button A with attachInterrupt.
2. In loop(), show the number on the LCD: Presses: N.
3. Write a second ISR for button B (GPIO 6): it sets presses back to 0. Attach it to button B.
4. Add delay(2000) to loop(), as in Task 7 of Lecture 5. Do you still lose any presses? Why not?

Solution: count the presses



```
void IRAM_ATTR onButtonA() { presses++; }
void IRAM_ATTR onButtonB() { presses = 0; }
void setup() {
  Wire.begin(8, 9);
  lcd.init(); lcd.backlight();
  pinMode(BUTTON_A, INPUT_PULLUP);
  pinMode(BUTTON_B, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(BUTTON_A),
                  onButtonA, FALLING);
  attachInterrupt(digitalPinToInterrupt(BUTTON_B),
                  onButtonB, FALLING);
}

void loop() {
  lcd.setCursor(0, 0);
  lcd.print("Presses: ");
  lcd.print(presses);
  lcd.print("   ");           // clear old digits
  delay(2000);
}
```

Question 3

A second ISR, `onButtonB`, sets `presses = 0`. It is attached to pin 6 with `FALLING`, like button A.

Question 4

No press is lost. The interrupt counts it even while `loop()` waits. The LCD just shows the new number up to 2 s later.

Compare with Task 7

The same `delay(2000)`, but now nothing is missed.

Two buttons at the same moment?

✓ On our ESP32-C3: safe

It has **one** core, and an ISR is not interrupted by another button. If A and B are pressed together, one ISR runs, then the other. Both presses are handled.

✓ Our Task 1: safe

loop() only **reads** presses. Only the ISRs change it.

But what if the processor had **two** cores?

What if we had two cores?

⚡ both buttons at the same moment



Suppose

Both buttons now count presses, and each ISR runs on **its own core**. Both buttons are pressed at the same moment, so both ISRs run **at the same time**.

presses was 5. Two presses. What should it be after both ISRs?

Two cores: a press is lost

	presses
before	5
expected	7
sometimes we get	6



One press is lost

Two cores ran `presses++` at the same moment, and the counter grew by **one**, not by two. Not every time: only when the timing is unlucky.

How can one line of code lose a press?

Keep this question. We meet it again at the end of today, and answer it on Tuesday.

Rules for an ISR: 1. keep it short

short ISR



long ISR



the main program waits

time

While the ISR runs

The main program is **suspended**. A long ISR stops the whole program.

Other interrupts wait too

A second press that comes during a long ISR is handled only **after** it.

A good ISR takes **microseconds**: presses++ and nothing more.

Rules for an ISR: 2. never wait

✗ Not allowed

```
● ● ●  
void IRAM_ATTR onButtonA() {  
    presses++;  
    delay(1000);    // never wait in an ISR  
}
```

✓ Allowed

```
● ● ●  
void IRAM_ATTR onButtonA() {  
    presses++;    // only record it  
}
```

delay() in an ISR

Depending on how `delay()` is made, it may not work in an ISR. On our ESP32 it does not: it does not wait at all, it stops `loop()` instead. And where it does work, it breaks rule 1. We explain the ESP32 case later, after FreeRTOS.

Try it: the example `ISRRule2` on [GitHub](#). In Wokwi, the ISR with `delay(1000)` took 17 μ s, and `loop()` stopped for 1 s.

Rule 1 is about **time**: be short. Rule 2 is about **what**: never call a function that waits.

Rules for an ISR: 2. watch out for hidden waiting

✗ Not allowed either



```
void IRAM_ATTR onButtonA() {  
    presses++;  
    lcd.print(presses);      // waits inside  
    Serial.println(presses); // waits inside  
}
```

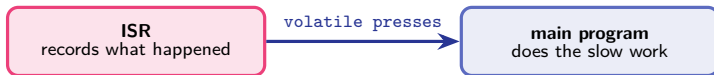
They wait, but you do not see it

lcd.print() waits with delayMicroseconds() and for the I2C bus.

Serial.println() waits until the serial port is free.

Be careful with library functions in an ISR.

Rules for an ISR: 3. share data carefully: volatile



The ISR writes

It only changes a variable: `presses++`.

loop() reads

It shows the value on the LCD, sends it, does the slow work. `volatile`: it reads the **fresh** value every time.

The ISR only **records** what happened. The main program **does the work**.

Rules for an ISR: 3. share data carefully: counting presses



```
volatile int presses = 0;    // the ISR adds 1
unsigned long counted = 0;   // all presses so far

void loop() {
    counted += presses;      // take the new presses
    presses = 0;             // start again from 0
    // ... the slow work ...
}
```

The idea

The ISR only counts the new presses in `presses`. `loop()` adds them to `counted`, the number of all presses so far, and sets `presses` back to 0.

In theory

Every press goes into `presses`, and from there into `counted`. No press is lost.

Now both the ISR and `loop()` **change** `presses`. What can go wrong?

Rules for an ISR: 3. share data carefully: a race

Step	Who	Runs	presses	counted	real presses
		start	2	10	12
1	main program	counted += presses;	2	12	12
2	ISR	presses++; (a new press)	3	12	13
3	main program	presses = 0;	0	12	13

⚠ A press between these two lines

The ISR came after `counted += presses`, but before `presses = 0`. `counted` is 12, but there were 13 presses: one is **lost**, even on one core. `volatile` does not help: it cannot stop the ISR between two lines. This is a **race**: we solve it on Tuesday.

Try it: the example `ISRRule3` on [GitHub](#). A timer makes 10 000 “presses” per second. How many are lost?

You can use interrupts. How does the processor do it?



Where to come back?

How does the processor know where the main program stopped?



The same registers

The ISR uses registers too. Why does the main program not notice?



When exactly?

Does the processor stop in the middle of an instruction?

To answer, we first need two tools: the **names of the registers** and the **stack**. Then we come back to interrupts and look inside the processor, step by step.

A register has two names

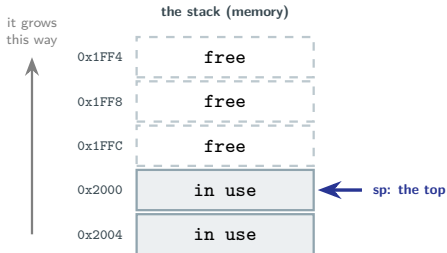
Number	Name	Used for
x2	sp	the stack pointer
x5	t0	temporary values
x6	t1	temporary values
x10	a0	arguments (addresses)
x11	a1	arguments (addresses)
x14	a4	arguments, temporary values
x15	a5	arguments, temporary values

The same register

t0 is x5. The processor only knows the number; the name tells people what the register is for. From now on we use the names.

`addi t0, t0, 1` is exactly `addi x5, x5, 1`.

The stack



The **stack** is a part of the memory where a program keeps values **for later**, for example the local variables of a function.

The top

The register *sp* (*stack pointer*) holds the address of the top. From the top down: **in use**. Above it: free.

It grows towards lower addresses

More values: *sp* goes **down**. Fewer: *sp* goes up.

A local variable lives on the stack

C



```
int f(void) {
    int a = 7;
    return a + 1;
}
```

One local variable

`a` exists only while `f` runs. Where does it live?

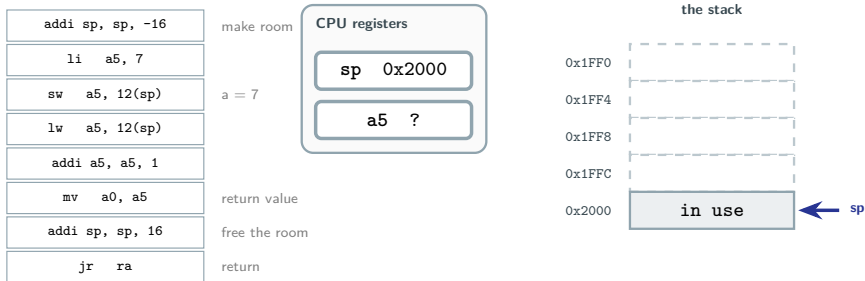
RISC-V (`gcc -O0 -fomit-frame-pointer`)



```
f:
    addi sp, sp, -16    # make room
    li   a5, 7
    sw   a5, 12(sp)     # a = 7, on the stack
    lw   a5, 12(sp)     # read a
    addi a5, a5, 1
    mv   a0, a5         # the return value
    addi sp, sp, 16     # free the room
    jr   ra             # return
```

The function takes room on the stack at the start, and gives it back at the end.

A local variable lives on the stack: step by step

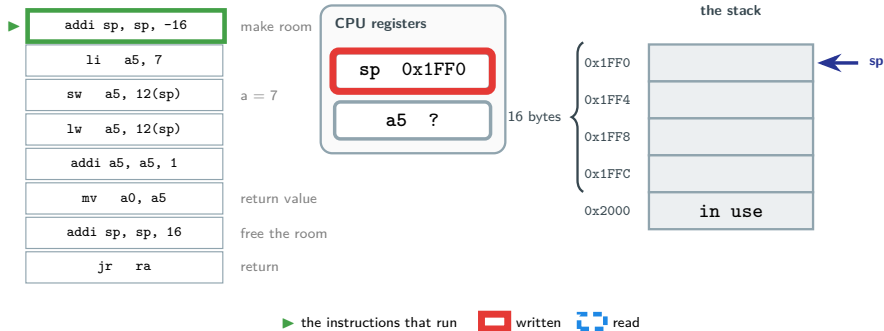


► the instructions that run  written  read

Before f

sp = 0x2000. Above it the stack is free.

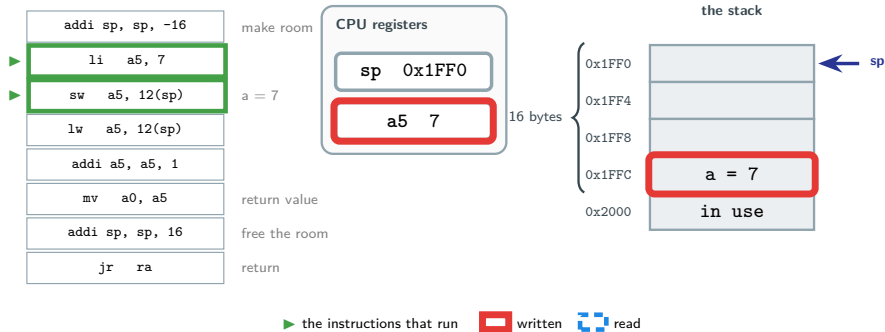
A local variable lives on the stack: step by step



`addi sp, sp, -16`

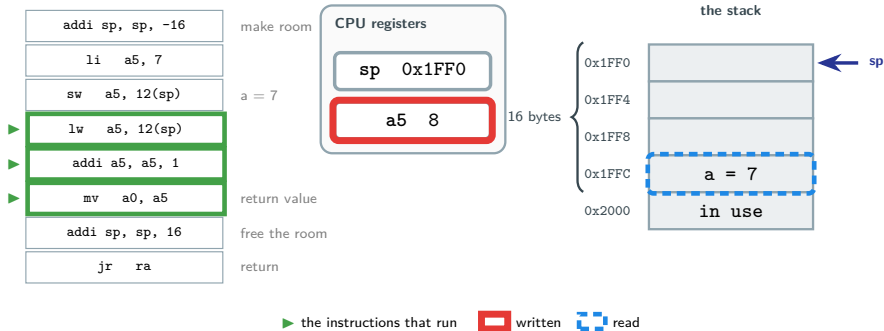
The stack **grows**: 16 bytes for f, `sp = 0x1FF0`. (RISC-V keeps `sp` a multiple of 16, so 16 and not 4.)

A local variable lives on the stack: step by step



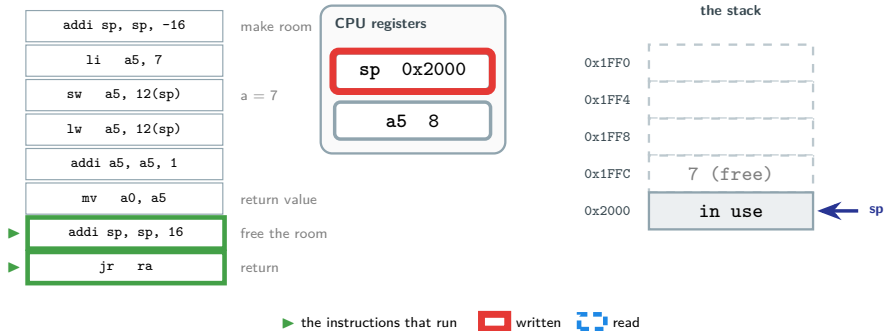
`li a5, 7` and `sw a5, 12(sp)`
 $a = 7$ is written at $sp + 12 = 0x1FFC$.

A local variable lives on the stack: step by step



`lw, addi, mv`
f reads a from the stack, adds 1 and puts the result (8) in a0.

A local variable lives on the stack: step by step



`addi sp, sp, 16`

The stack **shrinks**: `sp = 0x2000` again. `a` is gone: the 7 is still in the memory, but that place is free now.

Task: find the stack in Compiler Explorer



```
int sum3(int a, int b, int c) {  
    int s = a + b;  
    int t = s + c;  
    return t;  
}
```

Settings

godbolt.org, language C, compiler **RISC-V**
(32-bits) gcc, options -O0
-fomit-frame-pointer.

Task

Compile the function and read its assembly.

1. Which instruction makes room on the stack? How many bytes?
2. a, b, c arrive in a0, a1, a2. Where on the stack are they written?
3. Where are s and t kept?
4. Which instruction frees the room?

If you finish early: change -O0 to -O2. What happens to the stack?

Solution: find the stack in Compiler Explorer



```
sum3:
    addi sp, sp, -32
    sw    a0, 12(sp)    # a
    sw    a1, 8(sp)     # b
    sw    a2, 4(sp)     # c
    lw    a4, 12(sp)
    lw    a5, 8(sp)
    add   a5, a4, a5
    sw    a5, 28(sp)    # s
    lw    a4, 28(sp)
    lw    a5, 4(sp)
    add   a5, a4, a5
    sw    a5, 24(sp)    # t
    lw    a5, 24(sp)
    mv    a0, a5
    addi  sp, sp, 32
    jr    ra
```

Questions 1 and 4

addi sp, sp, -32 makes room for **32 bytes**; addi sp, sp, 32 frees it.

Questions 2 and 3

Variable	Comes from	On the stack
a	a0	12(sp)
b	a1	8(sp)
c	a2	4(sp)
s	a + b	28(sp)
t	s + c	24(sp)

Your offsets may differ a little.

If you finished early

With -O2: add a0, a0, a1, add a0, a0, a2, ret. No stack at all: the values stay in registers.

Who saves which register?

When a function f calls a function g , both use the **same** registers. RISC-V has a rule for it (the *calling convention*): which registers g may change freely, and which it must give back unchanged.

Registers	Who saves them	When
t0 to t6, a0 to a7	the caller f	before the call, only if f still needs them
s0 to s11, sp	the called function g	in its prologue, only if g changes them

g may use a5 freely

a5 is in the first group. If f still needs it after the call, f saves it on its stack before the call.

g must give back s1

If g wants to use s1, its prologue saves it and its epilogue restores it.

So a function prologue saves **only** the registers of the second group that the function changes.

Back to interrupts

A call is prepared

Before `f` calls `g`, `f` knows it, and saves what it still needs.

An interrupt is not

An interrupt can come between **any** two instructions.
The main program has no chance to save anything.

So an ISR must save **every** register it changes, from both groups, and restore it before it returns.

Inside the CPU: first a simplified ISR

Now: an ISR written by hand, in assembly (simplified)

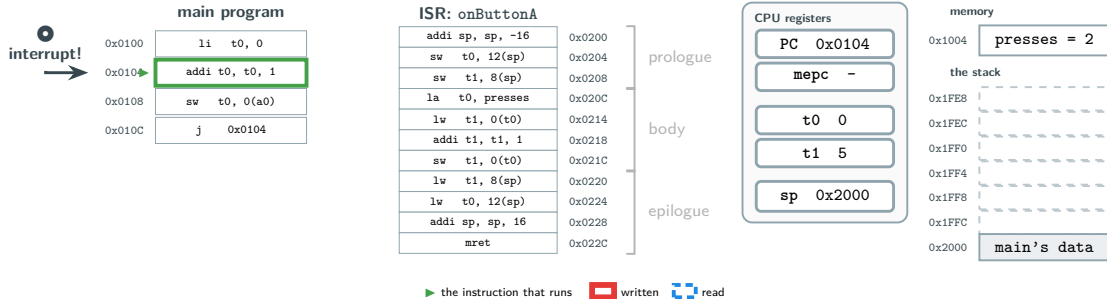
We write every instruction ourselves, and the processor jumps **straight** to it: no other code runs before it. So we can follow every register, step by step.

Later: our real ISR, written in C

`onButtonA` is C code. The compiler makes its machine code, and we will see what changes then.

First the simple case, then the real one.

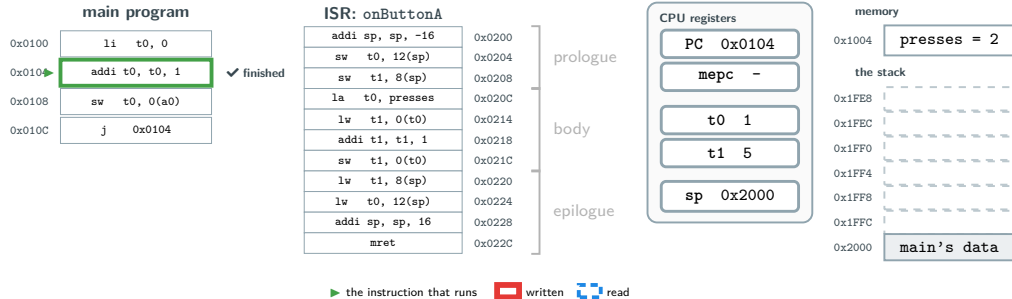
Inside the CPU: 1. the interrupt arrives



1 The signal

Button A raises an interrupt signal while the CPU runs addi at 0x0104.

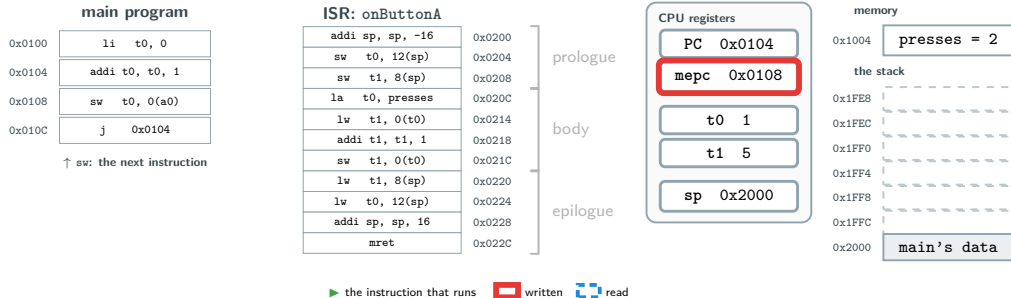
Inside the CPU: 1. the interrupt arrives



2 Finish the instruction

The CPU does not stop in the middle: it first **finishes** addi. Now $t0 = 1$.

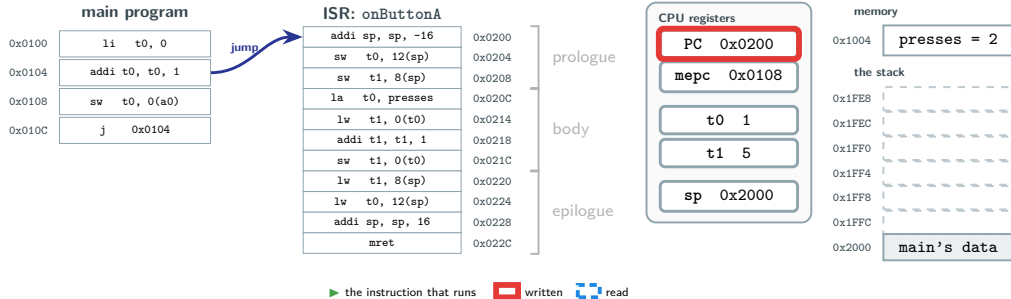
Inside the CPU: 1. the interrupt arrives



3 Remember the way back

The CPU writes into mepc the address of the **next** instruction, sw, at 0x0108. The instruction addi has already been executed.

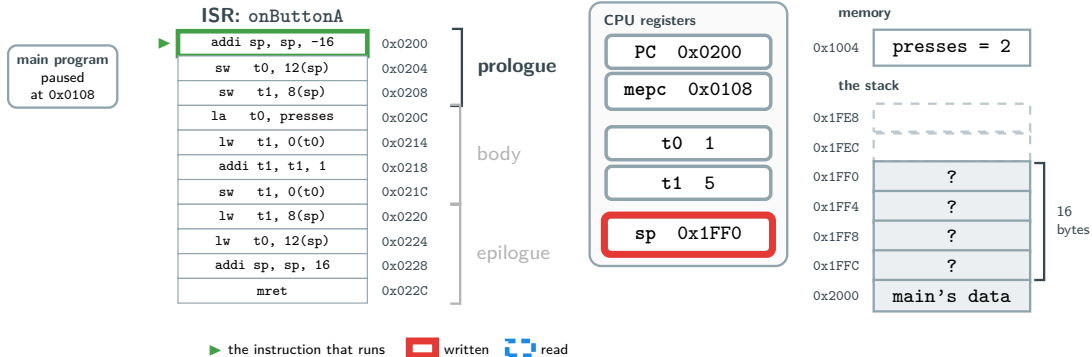
Inside the CPU: 1. the interrupt arrives



4 Jump

The PC gets the address of the ISR: 0x0200. The ISR starts with its **prologue**: it saves the registers it will change.

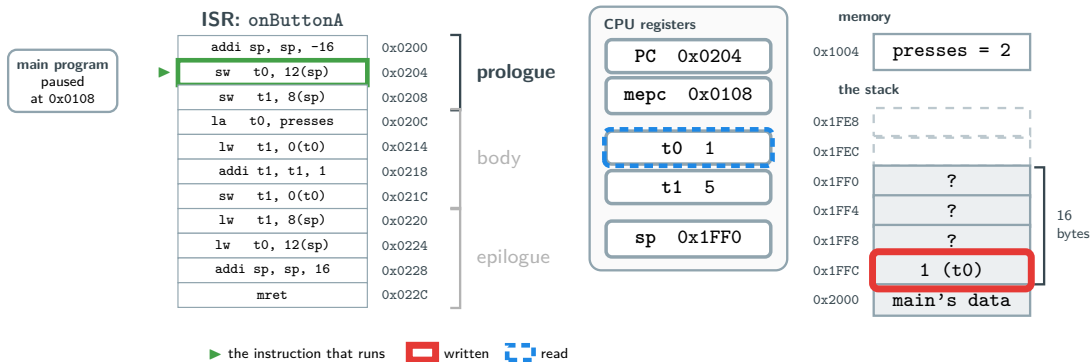
Inside the CPU: 2. the prologue



```
addi sp, sp, -16
```

Make room on the stack: 16 bytes, so that sp stays a multiple of 16. We need only 8 of them.

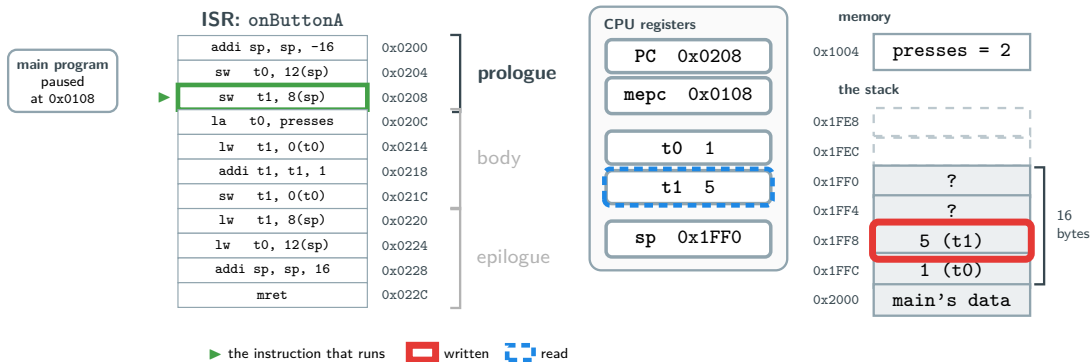
Inside the CPU: 2. the prologue



sw t0, 12(sp)

Save the main program's t0 (1): the body is about to use t0.

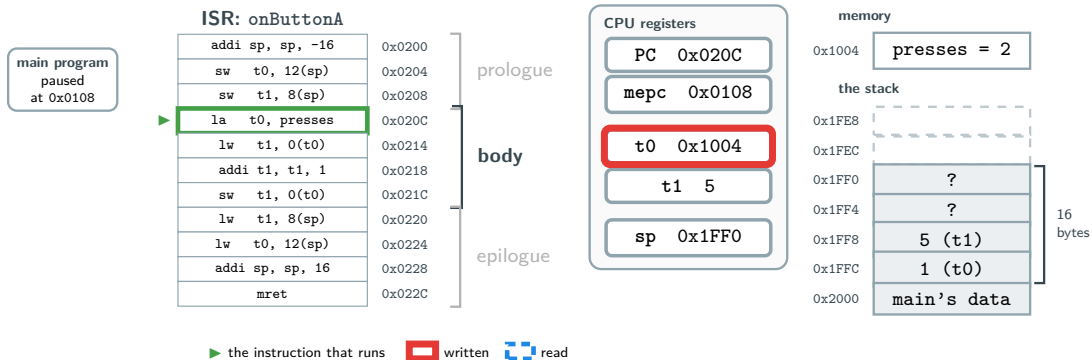
Inside the CPU: 2. the prologue



sw t1, 8(sp)

Save the main program's t1 (5): the body uses t1 too.

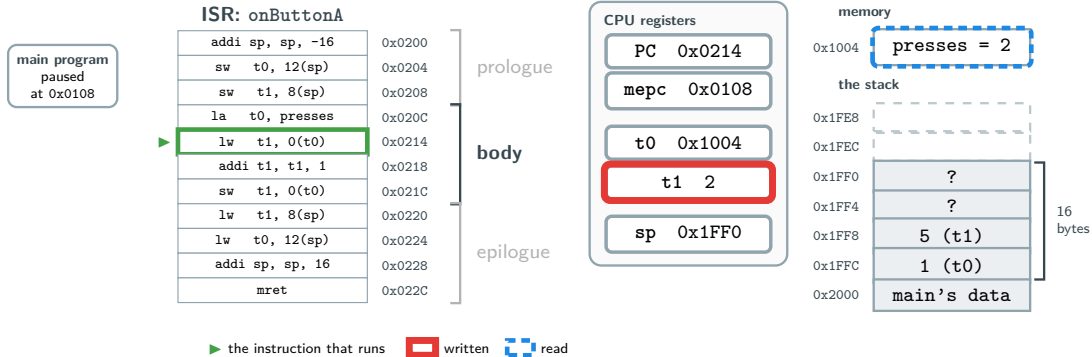
Inside the CPU: 3. the body



la t0, presses

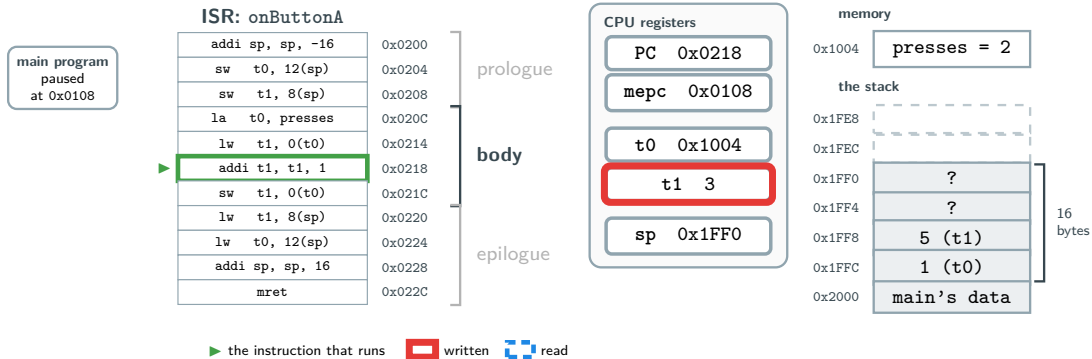
We load the **address** of the variable presses: it is stored at 0x1004, so t0 = 0x1004. (la is two real instructions, 8 bytes: the next one is at 0x0214.)

Inside the CPU: 3. the body



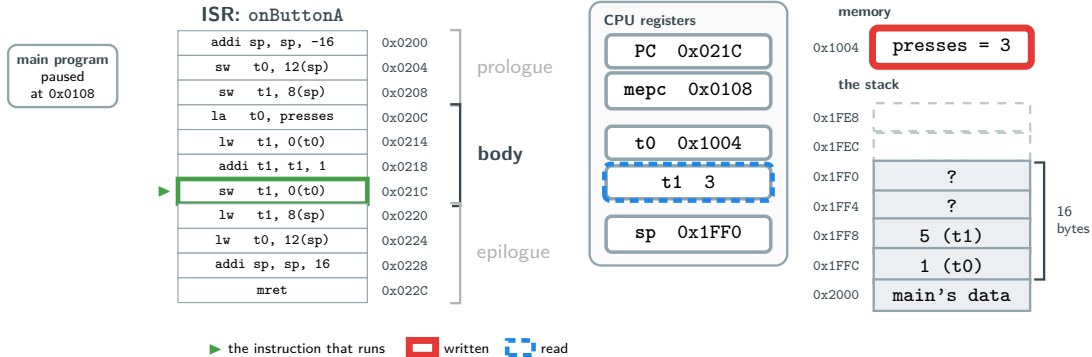
```
lw t1, 0(t0)
t1 = presses = 2.
```

Inside the CPU: 3. the body



```
addi t1, t1, 1
t1 = 3.
```

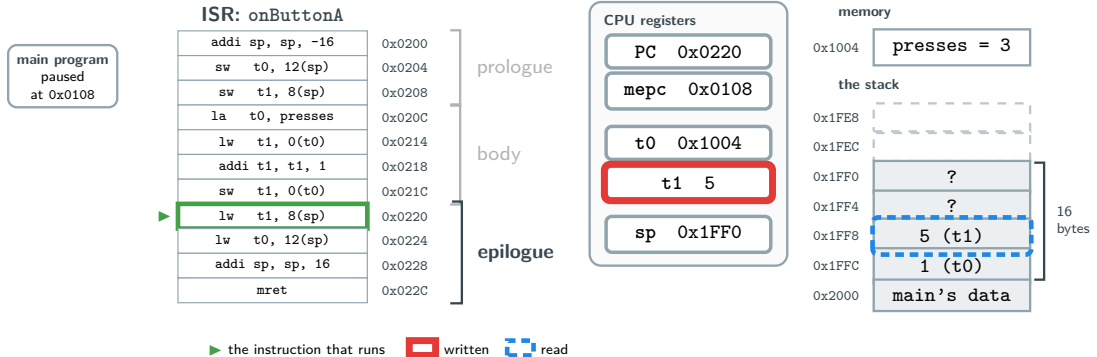
Inside the CPU: 3. the body



```
sw t1, 0(t0)
```

presses = 3. These four instructions are presses++: the work of the ISR.

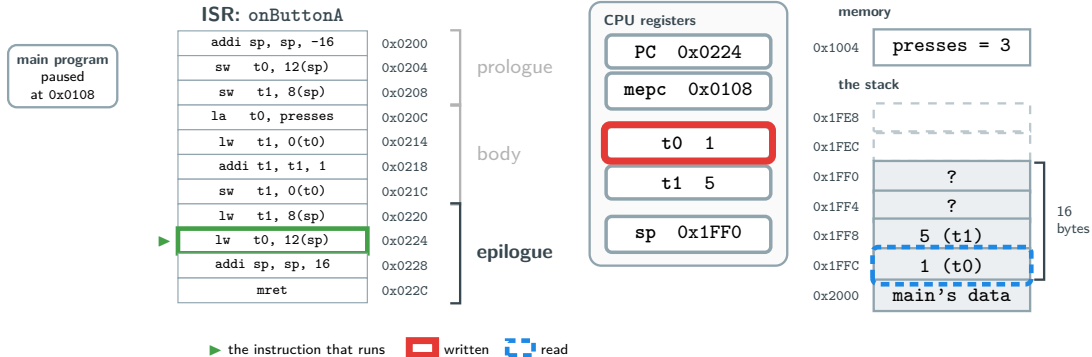
Inside the CPU: 4. the epilogue and return



```
lw t1, 8(sp)
```

Restore the main program's t1: 5.

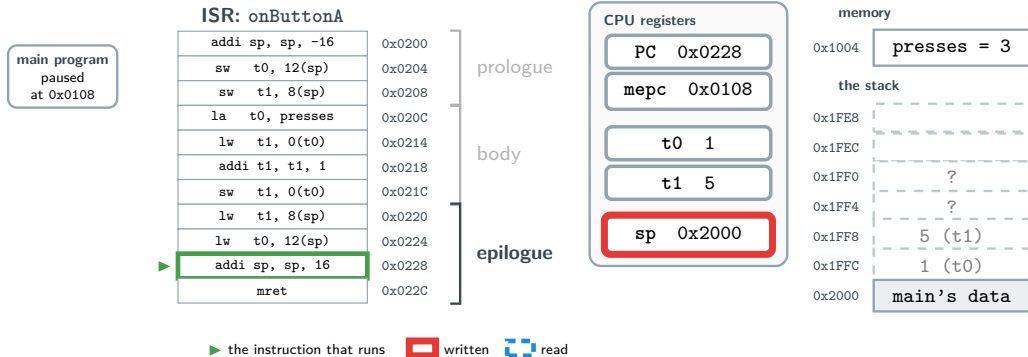
Inside the CPU: 4. the epilogue and return



lw t0, 12(sp)

Restore the main program's t0: 1.

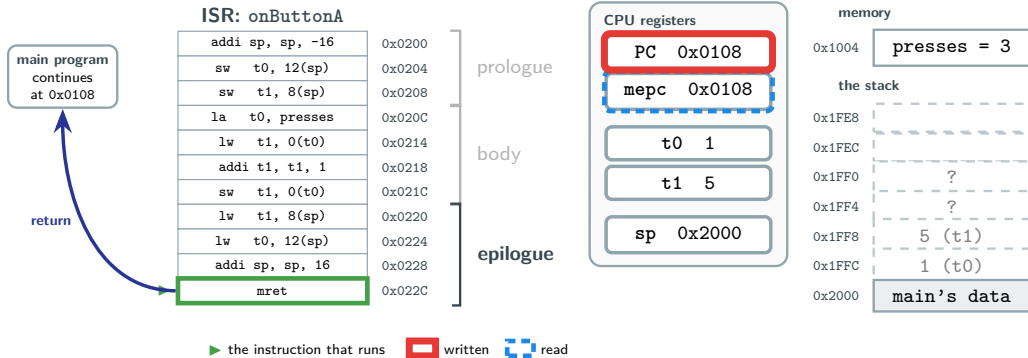
Inside the CPU: 4. the epilogue and return



```
addi sp, sp, 16
```

Free the room: sp is 0x2000 again. The stack is as before the interrupt.

Inside the CPU: 4. the epilogue and return



mret

PC ← mepc = 0x0108. The main program continues with sw, with t0 = 1 and t1 = 5: it **does not know** that it was interrupted.

Simulator 1: an interrupt step by step

Simulator task

Open **ComputerHardwareSimulators**, topic **Interrupt step by step**. In the list *The signal from button A arrives during the instruction at*, choose 0x0108.

```

● ● ●
0x0100  li    t0, 0
0x0104  addi  t0, t0, 1
0x0108  sw    t0, 0(a0)
0x010C  j     0x0104

```

1. **Predict** the saved PC (mepc). Then step through and check.
2. The ISR uses t0 and t1. What are they in the main program after the return?
3. Choose 0x0104 in the list. What is the saved PC now? Why?

[ksawerym.github.io/ComputerHardwareSimulators/#cc-irq](https://github.com/ksawerym/ComputerHardwareSimulators/#cc-irq)

Solution: Simulator 1

Question 1

0x010C. The CPU first finishes `sw`, so it saves the address of the **next** instruction.

Question 2

The same as before the interrupt: $t0 = 1$, $t1 = 5$. The ISR saved both on the stack and restored them before `mret`.

Question 3

0x0108: again the instruction **after** the one that was running.

The main program continues **exactly where it stopped**, with all its registers.

The rule: save what you change

The hardware saves **only the PC** (in mepc). Every register that the ISR changes, the ISR must **save** in its prologue and **restore** in its epilogue.

Our ISR

Its body changes t0 and t1. So the prologue saves exactly these two: that is enough. A normal function would not save them (first group), an ISR must.

sp?

It changes too, but the epilogue moves it back by the same 16 bytes.

Then the main program finds all its registers exactly as it left them.

But our ISR is written in C

Now the real case: our ISR is written in C. What does the compiler make of it?

We write

```
void IRAM_ATTR onButtonA() {  
    presses++;  
}
```

IRAM_ATTR

Not part of C: a name from the ESP32 libraries.

Try it: the example ISRinC on [GitHub](#).

The compiler makes (gcc -O2, presses at 0x1004 as before)

```
onButtonA:  
    lui    a4, 0x1           # a4 = 0x1000  
    lw     a5, 4(a4)         # a5 = presses (0x1000 + 4)  
    addi   a5, a5, 1  
    sw     a5, 4(a4)  
    ret                                # a normal return
```

A normal function

It changes a4 and a5 and **saves neither**. It ends with ret, not mret.

For a normal function this is correct: a4 and a5 are saved by the caller. But nobody **calls** an ISR.

Who saves the registers?



Why all registers?

This code does not know which registers our C function will use. So it saves all of them, a4 and a5 too.

Simplified

This process is a little simplified. The exact story is in the code of **ESP-IDF**, the ESP32 libraries: the link is on the next slide.

`onButtonA` does not need to save anything: the code from the ESP32 libraries has already done it.

Let us look at it in the library

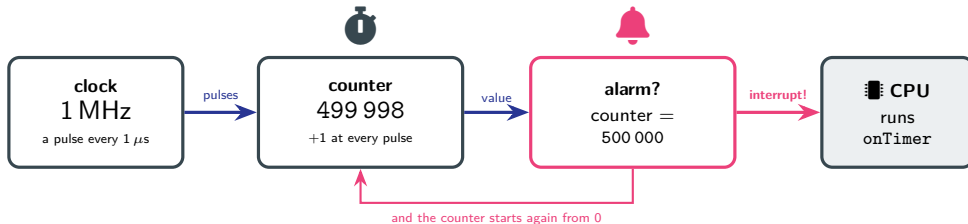
🔗 The code the CPU jumps to: `_interrupt_handler`

github.com/espressif/esp-idf/blob/v5.5.5/components/riscv/vectors.S#L385

Look for	What it does
<code>save_general_regs</code>	Makes room on the stack and saves all registers there, one sw for each.
<code>save_xepc</code>	Saves mepc: the address where the main program will continue.
<code>jal _global_interrupt_handler</code>	Finds out which interrupt it was and calls its function: in the end our <code>onButtonA</code> .
<code>restore_general_regs</code>	Loads all registers back from the stack.
<code>xret</code>	Here the same as <code>mret</code> : back to the main program.

We use ESP-IDF version 5.5.5.

A timer is a stopwatch with an alarm



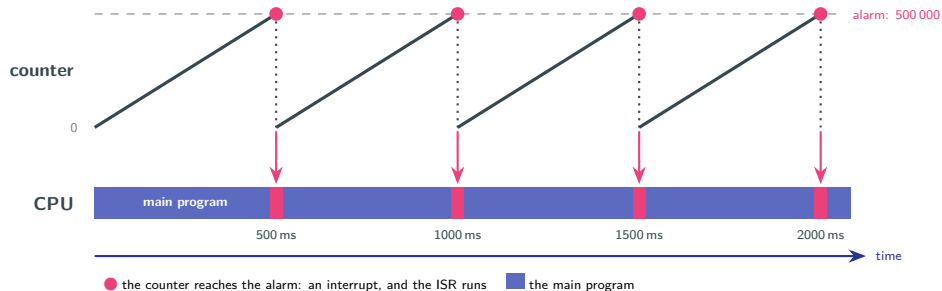
It counts by itself

A **timer** is a counter in hardware. A clock gives it pulses at a fixed rate, and at every pulse it adds 1. The CPU does not have to do anything.

It rings

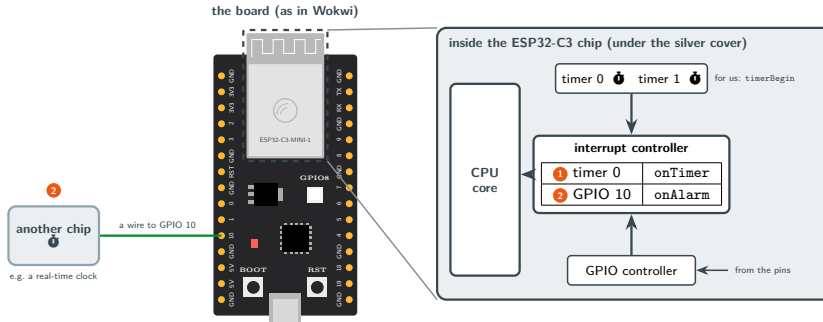
When the counter reaches the set value (the **alarm**), the timer raises an **interrupt**. Then it can start again from 0, and ring again.

Interrupts from a timer



A timer interrupt lets the processor do something **regularly**, without checking the time in the main program.

Where is the timer?



1 A timer inside the chip

It has its **own row** in the table of the interrupt controller. The CPU knows at once that the timer rang. Two timers are for us (`timerBegin`).

2 A timer on another chip

Its signal comes on a wire to a pin. For the CPU it is only “GPIO 10 changed”, like button A. It does not know that a clock is behind it.

A timer in the code

```

● ● ●
const int LED = 4;
hw_timer_t *timer = nullptr;
volatile bool ledOn = false;

void IRAM_ATTR onTimer() {           // the ISR
    ledOn = !ledOn;
    digitalWrite(LED, ledOn);
}

void setup() {
    pinMode(LED, OUTPUT);
    timer = timerBegin(1000000);
    timerAttachInterrupt(timer, &onTimer);
    timerAlarm(timer, 500000, true, 0);
}

```

The whole program

onTimer is an **ISR**: it switches the LED.
 setup() starts a timer and connects its interrupt to onTimer.

Three new things, one by one

timerBegin: the clock of our stopwatch.
 timerAttachInterrupt: which ISR runs.
 timerAlarm: when it rings.

A timer in the code: timerBegin

```
● ● ●  
  
const int LED = 4;  
hw_timer_t *timer = nullptr;  
volatile bool ledOn = false;  
  
void IRAM_ATTR onTimer() {           // the ISR  
    ledOn = !ledOn;  
    digitalWrite(LED, ledOn);  
}  
  
void setup() {  
    pinMode(LED, OUTPUT);  
    timer = timerBegin(1000000);  
    timerAttachInterrupt(timer, &onTimer);  
    timerAlarm(timer, 500000, true, 0);  
}
```

Take a timer and set its clock

The ESP32-C3 has **two** timers for us. `timerBegin` takes one that nobody uses yet (NULL if both are taken) and sets its clock to **1 000 000** pulses per second: the counter grows by 1 every $1\ \mu\text{s}$.

It returns the timer. We keep it in `timer`, of the type `hw_timer_t *`.

A timer in the code: timerAttachInterrupt

```
● ● ●  
const int LED = 4;  
hw_timer_t *timer = nullptr;  
volatile bool ledOn = false;  
  
void IRAM_ATTR onTimer() {           // the ISR  
    ledOn = !ledOn;  
    digitalWrite(LED, ledOn);  
}  
  
void setup() {  
    pinMode(LED, OUTPUT);  
    timer = timerBegin(1000000);  
    timerAttachInterrupt(timer, &onTimer);  
    timerAlarm(timer, 500000, true, 0);  
}
```

Which ISR?

Just like attachInterrupt for the button: it writes onTimer into the table, for the interrupt of **this** timer.

For the button we also gave a pin and FALLING. A timer needs neither: it has no pin and only one event, the alarm.

A timer in the code: timerAlarm

```

● ● ●
const int LED = 4;
hw_timer_t *timer = nullptr;
volatile bool ledOn = false;

void IRAM_ATTR onTimer() {           // the ISR
    ledOn = !ledOn;
    digitalWrite(LED, ledOn);
}

void setup() {
    pinMode(LED, OUTPUT);
    timer = timerBegin(1000000);
    timerAttachInterrupt(timer, &onTimer);
    timerAlarm(timer, 500000, true, 0);
}

```

When does it ring?

500000: when the counter reaches 500 000 steps of 1 μ s, so after **500 ms**.
 true: then start again from 0.
 0: and repeat without a limit.

The LED blinks with **no code in loop()**: the timer calls onTimer every 500 ms.

Try it: a timer blinks the LED

Task

Open the example `Timer` from [GitHub](#) (or paste the code into your project **Circuit**). Run it.

1. Does the LED on GPIO 4 change every 500 ms?
2. Change 500000 to 100000. What happens?
3. `loop()` is empty. Who switches the LED?

Solution: a timer blinks the LED

Question 1

Yes: on for 500 ms, off for 500 ms.

Question 2

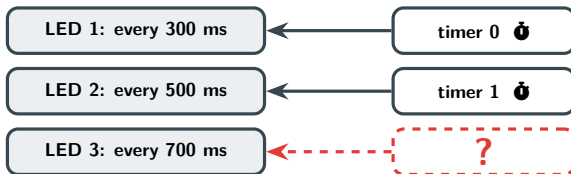
It blinks 5 times faster: the alarm rings every $100\,000\,\mu s = 100\text{ ms}$.

Question 3

The ISR `onTimer`. The timer interrupt calls it every time the alarm rings.

The CPU does not check the time at all: the timer tells it when.

Two timers, three LEDs?



What we saw

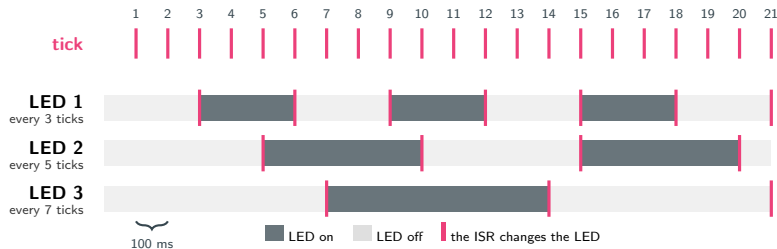
The ESP32-C3 has only **two** timers for us. One timer rings with one period.

The question

Do we need a third timer, on another chip, as on the slide “Where is the timer?”

No. **One** timer is enough, if it rings often.

The idea: one tick, many rhythms



One timer interrupt at a fixed rate is called a **tick**. Here: a tick every 100 ms. At every tick, the ISR **decides** what to do: LED 1 every 3 ticks, LED 2 every 5, LED 3 every 7.

The tick in the code



```
volatile unsigned long ticks = 0;
volatile bool led1 = false, led2 = false, led3 = false;
```

```
void IRAM_ATTR onTick() {           // every 100 ms
    ticks++;
    if (ticks % 3 == 0) { led1 = !led1; digitalWrite(LED1, led1); }
    if (ticks % 5 == 0) { led2 = !led2; digitalWrite(LED2, led2); }
    if (ticks % 7 == 0) { led3 = !led3; digitalWrite(LED3, led3); }
}
```

`ticks % 3 == 0`
Every third tick: 300 ms.

One timer

`timerAlarm(timer, 100000, true, 0):`
a tick every 100 ms.

Remember this idea: a **regular tick**, and a **decision** at every tick. We will need it in *The scheduler*.

Simulator 2: the tick

Simulator task

Topic **Timer and tick**: a tick every 100 ms. LED 1 changes every 3 ticks, LED 2 every 5, LED 3 every 7. At the start all LEDs are off. **Predict first**, then check.

1. Which LEDs change at tick **15**? At tick **21**? At tick **35**?
2. When are all three LEDs **on at the same time** for the first time?
3. When do all three **turn on in the same tick** for the first time? Calculate it with the least common multiple, then check.
4. All three change together again at the next such tick. Do they turn on, or off, then?

	LED 1	LED 2	LED 3
tick 15	...	...	...
tick 21	...	...	...
tick 35	...	...	...

Least common multiple (LCM): the smallest number divisible by all of them. Example: $\text{LCM}(4, 6) = 12$.

Solution: Simulator 2

Question 1: which LEDs change

	LED 1	LED 2	LED 3
tick 15	✓	✓	-
tick 21	✓	-	✓
tick 35	-	✓	✓

Question 2

After tick **9** (0.9 s), for one tick: LED 2 turns off at tick 10.

Question 3

$\text{LCM}(3, 5, 7) = \mathbf{105}$: tick 105, after 10.5 s. Each LED changed an **odd** number of times (35, 21, 15), so all three turn on.

Question 4

Tick 210, after 21 s. An **even** number of changes (70, 42, 30): all three turn **off**.

“On at the same time” and “turn on in the same tick” are different questions. Only the second one is the LCM.

Task 2: three LEDs



```
const int LED1 = 4, LED2 = 7, LED3 = 3;
hw_timer_t *timer = nullptr;
volatile unsigned long ticks = 0;
volatile bool led1, led2, led3;
```

```
// TODO 1: the timer ISR onTick()
```

```
void setup() {
    pinMode(LED1, OUTPUT);
    pinMode(LED2, OUTPUT);
    pinMode(LED3, OUTPUT);
    // TODO 2: timerBegin,
    // timerAttachInterrupt, timerAlarm
}
```

```
void loop() { } // nothing here
```

Task

Use your circuit with three LEDs or the template from [GitHub](#).

Make LED 1, 2 and 3 change every **300**, **500** and **700 ms** with **one** timer: a tick every 100 ms, as on the slide “The tick in the code”.

Run it: do the LEDs blink as in Simulator 2?

Solution: three LEDs



```
void IRAM_ATTR onTick() {           // every 100 ms
    ticks++;
    if (ticks % 3 == 0) { led1 = !led1; digitalWrite(LED1, led1); }
    if (ticks % 5 == 0) { led2 = !led2; digitalWrite(LED2, led2); }
    if (ticks % 7 == 0) { led3 = !led3; digitalWrite(LED3, led3); }
}

void setup() {
    pinMode(LED1, OUTPUT); pinMode(LED2, OUTPUT);
    pinMode(LED3, OUTPUT);
    timer = timerBegin(1000000);
    timerAttachInterrupt(timer, &onTick);
    timerAlarm(timer, 100000, true, 0); // 100 ms
}

void loop() { } // nothing here
```

The ISR

onTick from the slide “The tick in the code”.

The timer

The same three calls as for one LED. Only the alarm is shorter: $100000 = 100 \text{ ms}$, one tick.

Three rhythms, one timer, and loop() is still empty.

Simulator 3: could we do this without interrupts?

With millis() in loop()

```

● ● ●
void loop() {
  if (millis() - last1 >= 300)
    { last1 += 300; toggle(LED1); }
  // the same for LED 2 and LED 3
  showOnLcd();           // several ms
}

```

With a timer interrupt

```

● ● ●
void IRAM_ATTR onTick() { // every 100 ms
  ticks++;
  if (ticks % 3 == 0) toggle(LED1);
  // the same for LED 2 and LED 3
}
void loop() { showOnLcd(); } // several ms

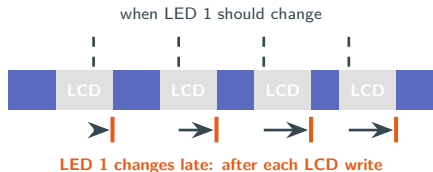
```

Simulator task

Topic **Timer and tick**, switch *How*: millis() in loop() or timer interrupt. In both, loop() also writes to the LCD.

1. millis() in loop(): look at the moments when LED 1 changes. Are they on time? How late at most?
2. Timer interrupt: are they on time now?
3. Both programs call showOnLcd(). Why does it hurt only one of them?

Solution: Simulator 3

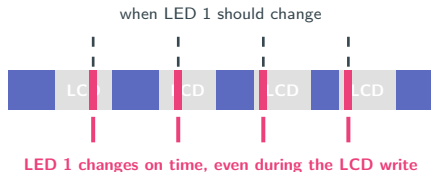


1. `millis()`

Almost, but **uneven**: LED 1 changes after 292 to 311 ms, up to about 20 ms late. That is one write to the LCD.

3. Why?

`millis()` looks at the clock only when `loop()` gets there, after the LCD write. The timer interrupt **stops** the LCD write, toggles the LED, and lets it continue.



2. Timer interrupt

Yes: exactly every 300 ms, 0 ms late.

`millis()` in `loop()` is polling again: the loop keeps asking "is it time yet?".

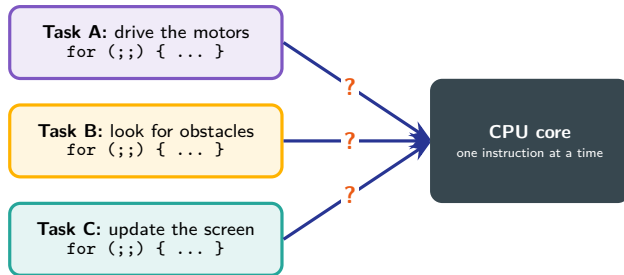
Interrupts: summary

Polling	Interrupts
the processor asks the device	the device tells the processor
short events can be missed	nothing is missed
wastes time on checking	the processor works until something happens
a rhythm is hard to keep	a timer gives an exact rhythm

Our tick decided **which LED** to change. Next: what if the tick decided **which program** runs?

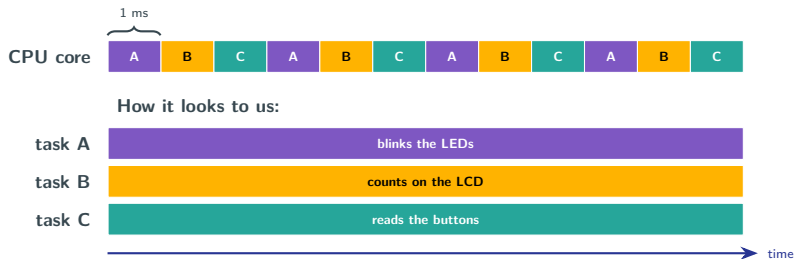
One processor, several tasks

In Task 2 each LED needed only a tiny piece of work, so it fit into the ISR. But think of a small **robot**: it drives, watches for obstacles and shows its state on a screen. Each of these is a **long program** with its own loop and waits. It cannot run inside a short ISR.



How can three programs, each with its own endless loop, share **one core**?

The idea: take turns very quickly



Time slice

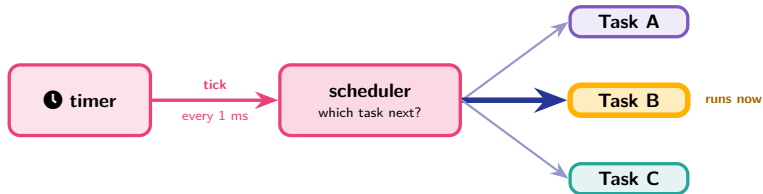
Each task runs for a short **time slice**, then the next one gets the core. The switches are so fast that all tasks seem to run at the same time.

Concurrency: several tasks make progress by **taking turns** on one core.

Several cores working at the same moment: *parallelism*. More on Tuesday.

Who decides when to switch?

The **scheduler** is the part of the system that decides **which task runs next** and switches to it.



Task 2
tick → **which LED?**

The scheduler
tick → **which task?**

The same idea as in Task 2: a regular **tick**, and a decision at every tick. On our ESP32 the tick comes every **1 ms**.

Simulator 4: time slices

Simulator task

Topic **Time slice**: tasks A, B and C take turns (round robin), one time slice = 1 ms. Task A starts at 0 ms. **Predict first**, then check.

1. Which task runs from **7 to 8 ms**?
2. How many task switches happen in **one second**?
3. Set the time slice to **500 ms**. Look at the row *how it looks to us*. Do the tasks still seem to run at the same time?

 ksawerym.github.io/ComputerHardwareSimulators/#cc-slice

Solution: Simulator 4



Question 1

B: A, B, C, A, B, C, A, B.

Question 2

1000: one switch every 1 ms.

Question 3

No. Now you can see them take turns: the LED stops for a whole second while B and C run.

Concurrency works because the time slices are **short**: much shorter than anything we can notice.

What does a task need to continue later?

If we stop task A now and resume it later, **what must we remember?**

What does a task need to continue later?

If we stop task A now and resume it later, **what must we remember?**



PC

where the task stopped: the next instruction

What does a task need to continue later?

If we stop task A now and resume it later, **what must we remember?**



PC

where the task stopped: the next instruction



Registers

the values it was working with, e.g.
t0, t1

What does a task need to continue later?

If we stop task A now and resume it later, **what must we remember?**



PC

where the task stopped: the next instruction



Registers

the values it was working with, e.g. `t0`, `t1`



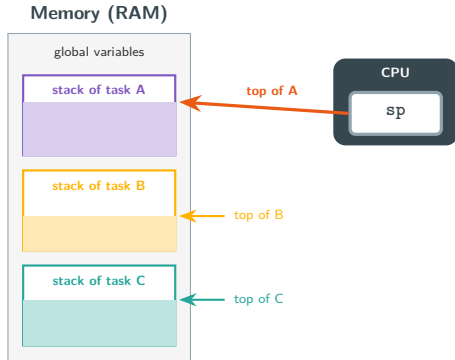
Stack

its local variables and the functions it is in

Together this is the task's **context**.

The interrupt entry code already puts **all** registers and `mepc` on the task's stack: that is its context.

Each task has its own stack



Its own stack

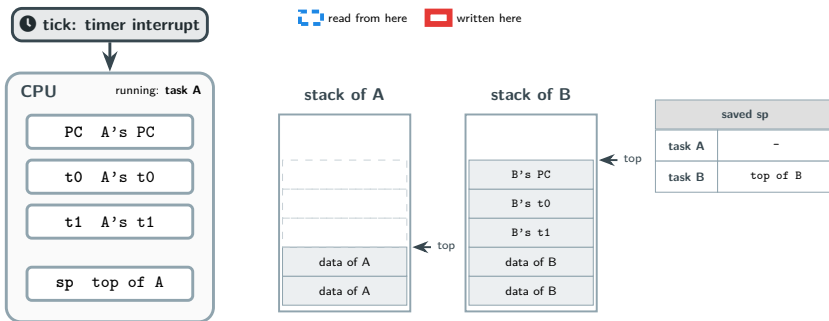
Each task has **its own stack** in memory: its local variables and saved registers.

The register `sp`

The stack pointer points to the stack of the task that is **running now**.

To switch tasks: save the registers on the old task's stack and **change** `sp`.

Context switch, step by step

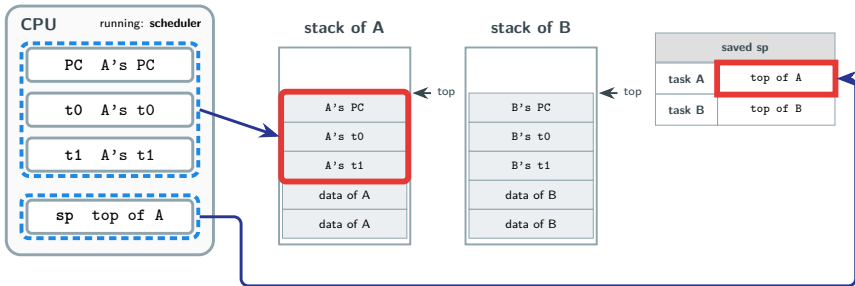


1 Tick

Task A runs. The tick arrives: $\text{mepc} \leftarrow \text{PC}$, $\text{PC} \leftarrow \text{mtvec}$. The CPU jumps to the interrupt entry code, not straight to the scheduler: the scheduler runs later, inside this interrupt.

Context switch, step by step

read from here written here

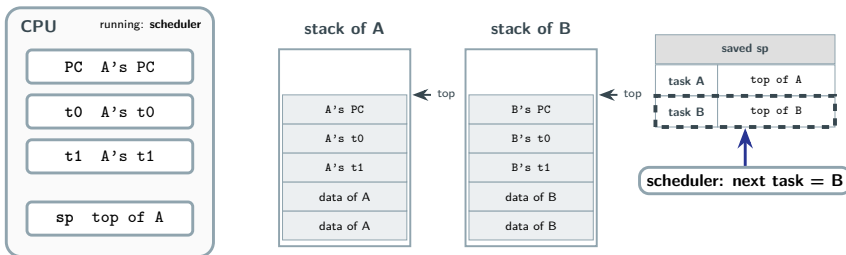


2 Save A

The entry code puts A's registers and its PC (from `mepc`) onto **stack A**, as we saw. The new top of stack A goes into the table of saved stack pointers.

Context switch, step by step

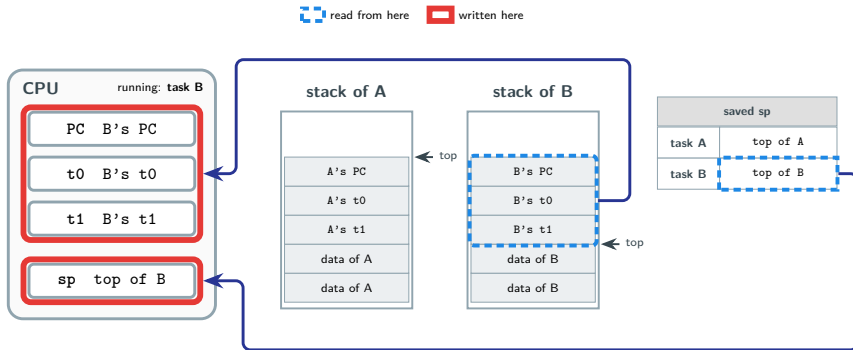
 read from here  written here



3 Choose B

Still inside the tick interrupt, the entry code calls the **scheduler**. It picks the next task: **B**.

Context switch, step by step



4 Restore B

$sp \leftarrow$ the saved sp of B. B's registers and PC come back from stack B: B continues as if nothing happened. These 4 steps are a **context switch**.

Simulator 5: a context switch

Simulator task

Topic **Context switch**. Before each step, predict what will change. Then click **Next** and check.

At the start

	CPU (task A runs)	saved on stack B
PC	0x0210	0x0308
t0	7	42
t1	3	9
sp	0x2000	saved sp of B: 0x2FF4

Each register takes **4 bytes**. Here a context = PC, t0, t1 (really: all registers).

1. Step 2, *Save A*: which values go onto stack A? What is the saved sp of A?
2. Step 4, *Restore B*: what are PC, t0, t1 and sp after the restore?
3. Where is A's t0 = 7 now? Who gives it back to A, and when?

 [ksawerym.github.io/ComputerHardwareSimulators/#cc-ctx](https://github.com/Ksawerym/ComputerHardwareSimulators/#cc-ctx)

Solution: Simulator 5

Question 1

0x0210, 7 and 3: 3 registers $\times$ 4 bytes = 12 = 0xC.

Saved sp of A: 0x2000 - 0xC = 0x1FF4.

Question 2

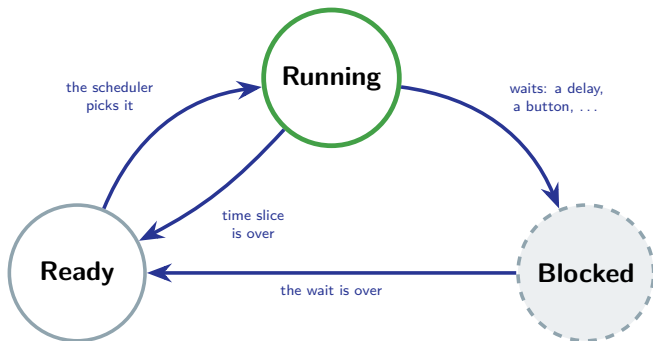
PC = 0x0308, t0 = 42, t1 = 9.
sp = 0x2FF4 + 0xC = 0x3000.

Question 3

On **stack A**. The scheduler restores it at the next switch back to A, exactly like B's values now.

Task A will never notice the break: its t0 waits for it on its own stack.

Task states



Running

On one core only **one** task runs at a time.

Ready

Could run, waits for its turn.

Blocked

Waits for something: a delay, a button. Gets **no** time slices.

Nobody is Ready?

Then the CPU runs a special task that does nothing: **IDLE**.

A Blocked task uses **no processor time**, unlike polling.

Which task runs next?

Round robin

Tasks with the **same priority** take turns, one time slice each.

Priorities

A Ready task with a **higher priority** always runs first. It even stops (preempts) a running task with a lower priority.

priority 2

H

when H is Ready, it runs first

priority 1

A

B

C

take turns: one time slice each

The scheduler picks the Ready task with the **highest priority**; equal priorities take turns.

Simulator 6: states, priorities and a shared counter

Simulator task

Topic **States and priorities**: A, B and C have priority 1. Task H has priority 2 and waits for the button.

1. At 2 ms, task C calls a 3 ms delay. In which state is C from 2 to 5 ms? Does it get any time slices?
2. Press the button: H becomes Ready. What happens to the task that is running?
3. What runs when A, B and C are all Blocked?

Then

Topic **Shared counter**: two tasks, each adds 1 to counter three times. The counter starts at 0.

4. **Predict** the final value.
5. Click **Run** three times and write down the results.



ksawerym.github.io/ComputerHardwareSimulators/#cc-states ksawerym.github.io/ComputerHardwareSimulators/#cc-shared

Solution: Simulator 6

Question 1

Blocked: no time slices. At 5 ms it becomes Ready again.

Question 2

It is stopped at once and goes to Ready. **H** runs. When H waits again, A, B and C continue.

Question 3

The **IDLE** task.

Questions 4 and 5

We expect **6**. But some runs give **4** or **5**, and the result changes from run to run.

⚠ Some additions were lost

Two tasks, six additions, and the counter did not count them all. Why? Keep this question: we will meet it again at the end of today.

A real scheduler: FreeRTOS

FreeRTOS is a small operating system for microcontrollers: mainly a **scheduler** plus tools that help tasks cooperate.

🔧 Free and open

Open source. Runs on many processors, including RISC-V.

🕒 RTOS

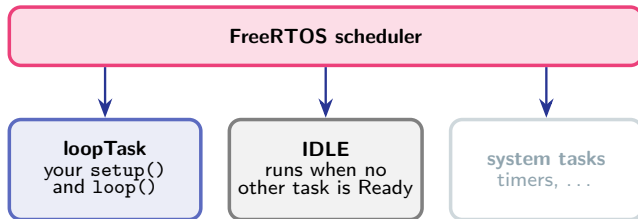
Real-time operating system: a task with a high priority reacts quickly.

📶 Everywhere

Sensors, home appliances, cars, ...

Everything from *The scheduler* is inside: the tick, time slices, states, priorities and the context switch.

You have used FreeRTOS all along



loopTask

On the ESP32, your `setup()` and `loop()` already run **inside a FreeRTOS task**.

`delay() = vTaskDelay()`

On the ESP32, `delay()` makes the task **Blocked**: while your `loop()` waited, it did not waste the CPU.

Nothing to install: we can create our own tasks.

Back to the ISR rule: `delay()` in an ISR

Now we can explain it

`delay()` is `vTaskDelay()`: it puts the **current task** to sleep. In an ISR, the current task is the one that was **interrupted**, here `loopTask`. So `loop()` sleeps, and the ISR goes on at once: it does not wait at all.

The example `ISRRule2` on [GitHub](#): the ISR with `delay(1000)` took $17\ \mu\text{s}$, and `loop()` stopped for 1s.

An ISR is not a task: the scheduler cannot put it to sleep. That is why an ISR must never wait.

Task 3: who runs loop()?



```
void setup() {  
  Serial.begin(115200);  
  Serial.print("Task: ");  
  Serial.println(pcTaskGetName(NULL));  
  Serial.print("Priority: ");  
  Serial.println(uxTaskPriorityGet(NULL));  
}  
  
void loop() {  
  delay(1000);  
}
```

Task

Any Wokwi project: replace `setup()` and `loop()` with this code (also on [GitHub](#)). Run it and open the **Serial Monitor**.

1. Which task runs `setup()`?
2. What is its priority?

New in this code

`pcTaskGetName`: the name of a task.

`uxTaskPriorityGet`: its priority. `NULL` means: the task that is running now. `Serial.print` sends text to the Serial Monitor.

Solution: who runs `loop()`?

Serial Monitor



Task: `loopTask`

Priority: `1`

Questions 1 and 2

The task `loopTask`, with priority `1`.

Next

The tasks we create will also get priority `1`, so they take turns with `loopTask`.

Creating a task

```
● ● ●  
void blinkTask(void *param) {  
    for (;;) {                               // forever  
        digitalWrite(LED1, HIGH);  
        vTaskDelay(pdMS_TO_TICKS(500));  
        digitalWrite(LED1, LOW);  
        vTaskDelay(pdMS_TO_TICKS(500));  
    }  
}  
  
void setup() {  
    pinMode(LED1, OUTPUT);  
    xTaskCreate(blinkTask, "blink", 2048, NULL, 1, NULL);  
}
```

The whole program

blinkTask blinks LED 1. setup() starts it as a new FreeRTOS task, next to loopTask.

Three new things

The task function, vTaskDelay and xTaskCreate. One by one.

Creating a task: the task function

```
void blinkTask(void *param) {  
    for (;;) { // forever  
        digitalWrite(LED1, HIGH);  
        vTaskDelay(pdMS_TO_TICKS(500));  
        digitalWrite(LED1, LOW);  
        vTaskDelay(pdMS_TO_TICKS(500));  
    }  
}  
  
void setup() {  
    pinMode(LED1, OUTPUT);  
    xTaskCreate(blinkTask, "blink", 2048, NULL, 1, NULL);  
}
```

A task is a function

A normal C function with one parameter, param: any value we want to give the task.

Inside: an **endless loop**, for (;;). A task never returns: when it is done, it deletes itself (Task 5).

Creating a task: vTaskDelay

```
● ● ●  
void blinkTask(void *param) {  
    for (;;) {                               // forever  
        digitalWrite(LED1, HIGH);  
        vTaskDelay(pdMS_TO_TICKS(500));  
        digitalWrite(LED1, LOW);  
        vTaskDelay(pdMS_TO_TICKS(500));  
    }  
}  
  
void setup() {  
    pinMode(LED1, OUTPUT);  
    xTaskCreate(blinkTask, "blink", 2048, NULL, 1, NULL);  
}
```

Wait without using the CPU

The task becomes **Blocked** for 500 ms. It gets no time slices, the other tasks run.

pdMS_TO_TICKS turns milliseconds into ticks (on the ESP32: 1 tick = 1 ms).
Arduino's delay() does exactly this.

Creating a task: xTaskCreate

```

● ● ●
void blinkTask(void *param) {
    for (;;) {                               // forever
        digitalWrite(LED1, HIGH);
        vTaskDelay(pdMS_TO_TICKS(500));
        digitalWrite(LED1, LOW);
        vTaskDelay(pdMS_TO_TICKS(500));
    }
}

void setup() {
    pinMode(LED1, OUTPUT);
    xTaskCreate(blinkTask, "blink", 2048, NULL, 1, NULL);
}

```

Start a new task

blinkTask: the task function
 "blink": a name, for debugging
 2048: its own stack, in bytes
 NULL: the value of param
 1: the priority
 NULL: a handle, not needed here

From now on the scheduler gives time slices to blinkTask and loopTask.

Task 4: three LEDs as three tasks

Task

The same three LEDs as in Task 2, but now **without the timer**. Template: [GitHub](#).

1. Remove the timer. Create **three tasks** with `xTaskCreate`: LED 1 changes every 300 ms, LED 2 every 500 ms, LED 3 every 700 ms.
2. Keep Loop: N on the LCD in `loop()`. Do the LEDs keep their rhythm?
3. In the task of LED 3, replace `vTaskDelay` with a **busy wait** (on the right). Do the other LEDs still blink? Now create this task with priority 2. What happens?

A busy wait for question 3



```
unsigned long t = millis();  
while (millis() - t < 700) { }
```

Hint

One function can serve all three tasks: pass the pin and the period through `param`.

If you finish early: replace `vTaskDelay` with `vTaskDelayUntil`. What is the difference?

Solution: three LEDs as three tasks



```
struct Blink { int pin; int ms; };  
Blink b1 = {4, 300}, b2 = {7, 500}, b3 = {3, 700};
```

```
void blinkTask(void *param) {  
    Blink *b = (Blink *)param;  
    pinMode(b->pin, OUTPUT);  
    for (;;) {  
        digitalWrite(b->pin, HIGH);  
        vTaskDelay(pdMS_TO_TICKS(b->ms));  
        digitalWrite(b->pin, LOW);  
        vTaskDelay(pdMS_TO_TICKS(b->ms));  
    }  
}
```

```
// in setup():  
xTaskCreate(blinkTask, "led1", 2048, &b1, 1, NULL);  
xTaskCreate(blinkTask, "led2", 2048, &b2, 1, NULL);  
xTaskCreate(blinkTask, "led3", 2048, &b3, 1, NULL);
```

Question 2

Yes. Each LED task is Blocked most of the time and gets the core within 1 ms when its delay is over.

Question 3, priority 1

The other LEDs still blink, but the busy task wastes its time slices: **polling again**.

Question 3, priority 2

The other LEDs and the LCD **stop**: the busy task is always Ready and has the highest priority.

If you finished early

vTaskDelayUntil wakes the task at fixed moments: small delays do not add up.

Two solutions of the same problem

	Task 2: one timer + ticks	Task 4: three FreeRTOS tasks
Who counts the ticks	you, in the ISR	the scheduler
The code of one LED	mixed into one ISR	its own simple loop
A fourth LED	change the ISR	one more xTaskCreate
Waiting	-	vTaskDelay: Blocked, no CPU
Cost	nothing extra	one stack per task (RAM)

Both use a timer tick. With FreeRTOS, the scheduler does the bookkeeping for you. Next: **look inside**, what did it do?

Look inside: a task in FreeRTOS

tasks.c (simplified)

```

● ● ●
typedef struct tskTaskControlBlock {
    volatile StackType_t *pxTopOfStack; /* MUST be first */
    ListItem_t    xStateListItem; /* Ready or Blocked list */
    UBaseType_t   uxPriority; /* priority */
    StackType_t *pxStack; /* start of the stack */
    char          pcTaskName[16]; /* name */
    /* ... */
} tskTCB;
    
```

pxTopOfStack

The **saved sp** from our context switch. It is the **first** field.

xStateListItem

On which list the task is: **Ready** or **Blocked**.

uxPriority

Its priority: 1 for our tasks.

TCB = Task Control Block: one per task. It is our table of saved stack pointers, and more.

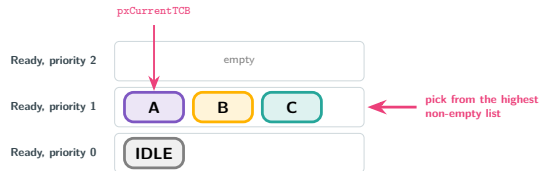
Choosing the next task: vTaskSwitchContext

tasks.c (simplified)

```

● ● ●
void vTaskSwitchContext( void )
{
    /* ... */
    taskSELECT_HIGHEST_PRIORITY_TASK();
    /* now pxCurrentTCB points to
       the TCB of the next task */
}

```



Step 3 of the context switch

The scheduler takes the first task from the **highest non-empty** Ready list and stores its TCB in `pxCurrentTCB`. Tasks with the same priority take turns.

Saving and restoring registers (simplified)

Save the old task (step 2)

```
● ● ●  
addi sp, sp, -CONTEXT_SIZE # room on its stack  
sw    ra, 1*4(sp)           # save registers  
sw    t0, 2*4(sp)  
...  
csrr  t0, mepc              # where it stopped  
sw    t0, 0(sp)             # save the PC  
lw    t0, pxCurrentTCB     # its TCB  
sw    sp, 0(t0)             # pxTopOfStack = sp
```

Restore the new task (step 4)

```
● ● ●  
lw    t0, pxCurrentTCB     # the new TCB  
lw    sp, 0(t0)            # sp = pxTopOfStack  
lw    t0, 0(sp)            # its saved PC  
csrw  mepc, t0  
lw    ra, 1*4(sp)          # restore registers  
lw    t0, 2*4(sp)  
...  
addi  sp, sp, CONTEXT_SIZE  
mret                          # continue the task
```

Exactly our 4 steps, in code: sw and lw for the registers, and one TCB per task.

The context switch on the ESP32-C3: the real code

```
● ● ●
_interrupt_handler:           # the tick arrives
    addi sp, sp, -128
    sw    ...                 # 2 all registers of A
    call rtos_int_enter       # on stack A
        lw a0, pxCurrentTCB
        sw sp, 0(a0)          # A's top = sp
    jal _global_interrupt_handler
    call rtos_int_exit
        call vTaskSwitchContext # 3 choose B
        lw a0, pxCurrentTCB
        lw sp, 0(a0)          # 4 sp = B's top
    lw    ...                 # all registers of B
    addi sp, sp, 128           # from stack B
    mret                      # B continues
```

The same entry code

It is `_interrupt_handler` from *Interrupts* inside the processor, with the inside of `rtos_int_enter` and `rtos_int_exit` shown below them (indented).

The trick

The entry code **always** saves all registers on the stack of the task that was interrupted. So to switch, it is enough to change `sp` before the epilogue: the epilogue then restores **B**.

A context switch is an interrupt that returns to a **different** task.

Simulator 7: the context switch with FreeRTOS names

Simulator task

Topic **Context switch** again, the same start as in Simulator 5. Turn on *FreeRTOS names*. **Before each step**, predict, then click **Next**.

1. Before step 2: to which TCB does `pxCurrentTCB` point?
2. After step 2: what is the value of `pxTopOfStack` in the TCB of A?
3. Step 3: what does `vTaskSwitchContext` change?
4. Step 4: which line of the assembly gives `sp` its new value? What is that value?

 [ksawerym.github.io/ComputerHardwareSimulators/#cc-ctx](https://github.com/Ksawerym/ComputerHardwareSimulators/#cc-ctx)

Solution: Simulator 7

Question 1

To the TCB of **A**: A was running.

Question 2

0x1FF4: the same saved sp as in Simulator 5.

Question 3

Only pxCurrentTCB: now it points to the TCB of **B**.

Question 4

lw sp, 0(t0), with t0 = the TCB of B: sp = 0x2FF4.
After the registers are restored, sp = 0x3000.

The table of saved stack pointers from *The scheduler* is the field pxTopOfStack in every TCB.

Task 5: a race between two tasks



```
const int N = 1000000;
volatile int counter = 0;
volatile bool doneA = false, doneB = false;

void incTask(void *param) {
    for (int i = 0; i < N; i++) {
        counter++;
    }
    *(volatile bool *)param = true;    // done
    vTaskDelete(NULL);                 // end this task
}
```

Task

Open the template from [GitHub](#). Two tasks increase one shared counter.

1. Start **two** incTask tasks with the same priority, with &doneA and &doneB as parameters.
2. In loop(), wait until both are done. Then show on the LCD: Goal: 2000000 and Got: N.
3. Run it several times. Is N always the same? Is it ever correct?

If you finish early: set $N = 1000$. Is the result correct now? Why?

Solution: a race between two tasks



```
void setup() {
    // the LCD as before
    xTaskCreate(incTask, "incA", 2048,
                (void *)&doneA, 1, NULL);
    xTaskCreate(incTask, "incB", 2048,
                (void *)&doneB, 1, NULL);
}

void loop() {
    if (doneA && doneB) {
        lcd.setCursor(0, 0);
        lcd.print("Goal: ");  lcd.print(2 * N);
        lcd.setCursor(0, 1);
        lcd.print("Got: ");   lcd.print(counter);
        while (true) delay(1000);
    }
    delay(10);                // wait without wasting the CPU
}
```

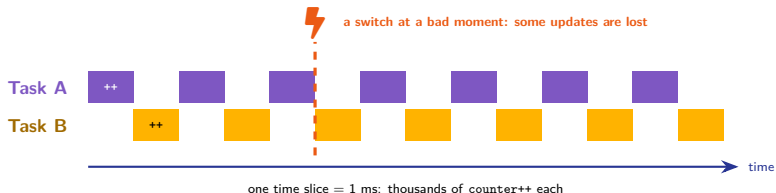
Question 3

N is **smaller** than 2 000 000 and **different** every time. It is never correct.

If you finished early

With N = 1000 each task finishes within one time slice: no switch comes in the middle, so the result is correct.

What happened?



This is a **race condition**: the result depends on **when** the scheduler switches the tasks.

We saw it three times

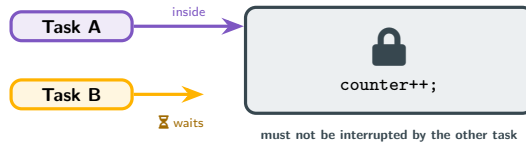
Two cores: two presses, presses grew by one. Simulator 6: 6 additions, counter 4 or 5. Task 5: two million additions, the counter got less.

The question

Why? `counter++` is **one line**. How can a switch fall *in the middle* of it?

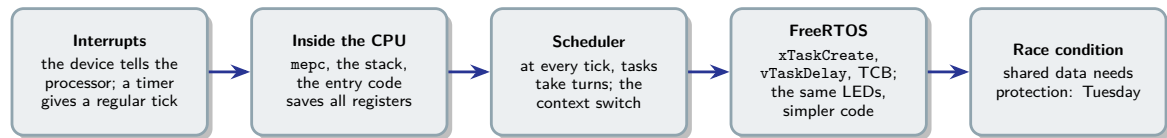
How do we fix this?

We need a way to say: *while I update the counter, nobody else may touch it.*



Tomorrow: **why** updates are lost, and how to fix it with **threads, mutexes and atomic operations**, in C++ on your laptops.

What we saw today



A brief history of concurrency: it all started with a button.

Check yourself

1. Why does an interrupt not miss a short press, while polling can?
2. What does the CPU save when an interrupt arrives, and why does the main program not notice the break?
3. Why does an ISR written in C not save the registers it uses, and who saves them?
4. What does `timerAlarm(timer, 500000, true, 0)` do, if the timer counts at 1 MHz?
5. What is a context switch? What is saved, and where?
6. Why does a task that waits with `vTaskDelay` not waste CPU time, while a busy loop does?

Try to answer each question in **2 or 3 sentences**.

Before Tuesday

From Tuesday we write **C++** on your own laptops. Install a compiler with **C++20** support (g++, clang or MSVC). Instructions will be sent today.

hello.cpp: check your installation

```
● ● ●  
#include <iostream>  
#include <thread>  
  
int main() {  
    std::thread t([] {  
        std::cout << "Hello from a thread\n";  
    });  
    t.join();  
}
```

Compile and run

```
● ● ●  
g++ -std=c++20 -pthread hello.cpp -o hello  
./hello
```

If it prints Hello from a thread, you are ready for Tuesday.